DS314: GENERATIVE AI & LLM SYSTEMS · ACADEMIC LAB MODULE

DS314 - Topic 3: LangChain Models & Autonomy Levels

Format:	Jupyter Notebook (.ipynb)	**Source:**	DS314 - Topic 3 - LangChain Models & Autonomy Levels.ipynb	**Render Engine:**	KaTeX + MathML (Zero-Delim Math)	**Academic Level:**	3rd Year BS Data Science

DS314 — DS Elective 1 | Generative AI

Week 3 - LangChain Models & Autonomy Levels

Intended Learning Outcome (ILO): Build a working LLM application using prompt templates, LCEL, embeddings, retrieval, conversation memory, and agentic tool use.

Each stage adds exactly one new idea on top of the last one:

Along the way you'll build:

- A minimal call to a Gemini chat model through LangChain
- A reusable, template-driven chain using LCEL (LangChain Expression Language)
- A retrieval-augmented generation (RAG) system backed by a small knowledge base
- A conversational assistant that remembers earlier turns
- An agent that decides on its own whether it needs to look something up

⚠ Before you code: setting up your tools

Before we build anything, we need to prepare the tools our application will use. Modern AI applications are rarely built from scratch — instead, we combine specialized libraries that each handle one part of the workflow: LangChain for orchestration, LangGraph for stateful and multi-step applications, an embeddings library for semantic search, a vector store for storing and searching those embeddings, and a model API for generating text.

By the end of this setup, you should be able to name each of these pieces and say, roughly, what job it does — even before you've seen them connected together.

What you'll need:

- A free Google AI Studio API key (https://aistudio.google.com/apikey), which you'll store as `GOOGLE_API_KEY` in a local `.env` file or your environment.
- Python and VS Code with the Jupyter extension installed.
- The packages installed below.

1. Create or activate your virtual environment:

Before installing the dependencies, **create or activate the virtual environment for this module**. A virtual environment keeps this project's Python packages isolated from other Python projects on your computer.

If you are starting from scratch, create a virtual environment in the same folder as this notebook:

`python -m venv .venv`
If you already created the `.venv` environment for this module, simply activate it before continuing.

2. Select the virtual environment in VS Code:

Open this notebook in VS Code and select the `.venv` environment as your Python interpreter/kernel using `Python: Select Interpreter` or the notebook's kernel selector.

Make sure the selected interpreter points to the same `.venv` environment that you activated above.

3. Install the dependencies:

Once the virtual environment is activated, install the packages required for this module.

Option 1 — Install directly from the notebook:

Run the `%pip install` cell below.

In [1]:
```
%pip install -q -U "langchain>=1.0" "langchain-core>=0.3" "langchain-google-genai" \
    "langchain-huggingface" "langchain-community" "langgraph>=0.2" \
    "transformers>=4.44" torch accelerate sentencepiece \
    "sentence-transformers>=2.6" faiss-cpu "python-dotenv"
```

Out [1]:
```
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 26.1.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Option 2 — Install from the terminal:

From a terminal, in the same folder as this notebook, run:

```
python -m pip install -r requirements.txt
```

⚠ Checkpoint: what does the device check below have to do with an LLM app?

This time we're stepping outside the LLM workflow itself to look at the computing environment. The cell below checks whether your machine can access a GPU through PyTorch. It won't power any of the Gemini API calls in this notebook — those run on Google's servers, not your laptop — but understanding the difference between **local computation** and **cloud-based model APIs** matters the moment you work with a model that runs on your own hardware, or with heavier workloads that need acceleration.

After this section, you should be able to: explain, in one sentence, the difference between calling a hosted model API and running a model locally on a GPU.

In [2]:
```
import torch

print("Torch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", DEVICE)
```

Out [2]:
```
Torch: 2.14.0+cpu
CUDA available: False
Using device: cpu
```

Section 1 — LangChain Models Module Concepts (Concept + Code)

1.1 What the Models module covers

- **LLMs (text-completion):** prompt in → text out. Good for drafting, Q&A, summarization, transformation.
- **Chat Models (multi-turn dialogue):** manage roles (system/user/assistant), keep conversational context.
- **Embeddings (vectors):** numerical representations for semantic search, retrieval, clustering.

LangChain provides consistent wrappers so you can swap providers without rewriting your application logic. That consistency is a big part of why LangChain is one of the most widely used LLM frameworks today: the same `.invoke()` / `.stream()` / `.batch()` interface works whether the model underneath is Gemini, a Hugging Face pipeline, or something else entirely.

1.2 Minimal LLM with LangChain

Checkpoint: what are we adding, and why?

This is our starting point: getting a model to respond from inside a Python application. You may already be used to talking to an AI model directly through a chat interface. Here, instead, we're programming an application that talks to the model for us. You'll see how an API key, a model configuration, a prompt, and a response all fit together — this is the foundation every other section in this notebook builds on. Rather than a person typing into a chat window, we're beginning to build *software* around a model.

What we're using: Gemini, accessed through Google AI Studio. `llm` below is a hosted chat model — every call goes out over the network to Google's servers and comes back with a response; nothing runs on your own machine. Chat-model support for Gemini lives in its own package, `langchain_google_genai`, separate from the core `langchain` package.

After this section, you should be able to: create a `ChatGoogleGenerativeAI` instance, call `.invoke()` on it with a plain string prompt, and read the text out of the response.

In [10]:

```
import os
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI

# Put the Google AI Studio key in a local .env file as GOOGLE_API_KEY=...
load_dotenv()

if not os.getenv("GOOGLE_API_KEY"):
    raise ValueError(
        "Set GOOGLE_API_KEY in the environment or a local .env file before running this cell."
    )

# A current Gemini Flash model.
MODEL_ID = "gemini-3.6-flash"

llm = ChatGoogleGenerativeAI(
    model=MODEL_ID,
)

# Try it!
prompt = "List three practical uses of large language models in daily life."

response = llm.invoke(prompt)

# response is an AIMessage object, not a plain string.
# .text gives us the generated text directly.
print(response)
```

Out [10]:

content=[{'type': 'text', 'text': 'Here are three practical uses of large language models (LLMs) in daily life:\n\n1. **Writing and Editing Assistance** \n   LLMs act as real-time writing partners. You can use them to draft professional emails, rewrite text for a specific tone (e.g., making a message sound more polite or concise), proofread documents for grammar, or generate creative ideas for blog posts and cover letters.\n\n2. **Personalized Learning and Problem-Solving** \n   LLMs can function as 24/7 personal tutors. If you encounter a complex topic, a foreign language phrase, or a difficult concept, you can ask the model to explain it at your specific knowledge level (e.g., "Explain how inflation works like I\'m 10 years old") or help guide you through step-by-step math problems.\n\n3. **Daily Planning and Task Organization** \n   LLMs excel at synthesizing information to help organize your life. You can ask them to generate customized travel itineraries based on your budget and preferences, create weekly meal plans complete with grocery lists, or suggest recipes using only the ingredients currently in your fridge.', 'extras': {'signature':
'EvcPCvQPARFNMg9GjaK9h5ZirO/wxoMYotXEa6NYHR6xxiw299ucj6dKbT/cR6OiIrpS/kR2fBlcanUPoIqFexqrL9I9Impk/XdAhPk
KHQfdP/YB2Ym+7uP54XLGCpINVTcTBCZUE2VPKGAtY/sBK+sAEdE97GbxWz4Jqy+XNF6CZchEJ+3ZGTQ0g8tSObN7mQUuatOldmQd92a
nCJ38Bd17l2U9SHatePtD12V6SqjuDE8LYqx41x2ZorRJljgFkArgHV1XAkPHQb9oLOwmLQrbWRBvDQ7XQSCB3hUyeZi+6yZazibBs5R
fhAMYBDbqRZ3x2W+yH/2njT5g36p+2XoUM1vQh/C8bTwEv+Pvy7Krqw12cH6VzVKI8Hl/e/N6edKbJcu2sPwRUYLqQM5hDHHojT32q9l
ky0aVz2Ieb1v3ELOpHnWBIu97gZ8jjEMfr4DJCGKuqt/m7MRZYzfPkuj47hpZN7E+txGGxVcIXLXpcKYzgo+P9BFZLxEf4XD57dWbX3L
psrs5HCDUjBFHAwzS8FVyJoG3iRHSrtmivboilES8kiWgi1jRajyEVHYAHlS/O2m3PWbmVtAqwBn96Kl8yHE/JJBsvMPHGN3ItdFLdbo
02LKt9d8U4EoK/UkFNSvYe3XYCIQ5+Y2iXLppUwWI5JFbEGTirZtgz+8DihFGhKI02MFU4eiPzdhqK0+9kwrjUcGir4y5ZRKKHHGiRrg
XIDLhGk+Lncc3oyJCLKIJz0TcL8hCzBelDx9iWybM6E6ICcOYjnZy57488UTdlVz0m/uggjznK0L0rUevRux8jsybhe+o4EUh/8ZS2Ke
b4QDxGBIAYI8VgPZU0P13+d32BisBMB2rWJV3gowlL9OVEY2IEsXc+3D2anNQ6pf7hpY2cuDQWnfCEd8/eQ1H3YmeWO4dP+ingmnu3dD
KjwCZeH7JM9b3F6LIrUdFXy4sil8BAJV8D1PdHPoOJcFPKJFCUdnhGf/d1GKQ7jppKYrz9BmLkQl7VvtPqbmhwHRvHAdEnXrlsvaM9Mz
xW3lzu4pTej28CDYKbk+u22jqhmToLlO5j/5DUVcz9P8xeAI53aUAFyGLFRiuC9D0ADG1dxl3Bz1knTec2GOrsl/hkw0xlP84Y7ObSgt
diUix7MRiZd61iHF/RunnfagqeCyV4P3LdAHZlPo4pteGnEslCeNaKpdDC2+5rLUxoapVliZBPfBQvcEP/jdfesMW54kb+mpEsMxVCJV
wC7SSOHbzerO7m4AhQhUFpkJ/TrKeMc0kGElG21YKblA31ZwpGTqt+FM3C53kymireVusTSyneAu77L33Be+exAXqfn++yXhoggN3InQ
0xtfuOervRk0hg6tdJGuRFGX3S3N5CJXa176dqh0hxf31xAmF+su0QBrR0bgKxlFhOAme+jCk19J8gfjztsH5qSIqOYXfJr+PTy2fjsc
56dIQwrl4YIxNzV2eYXNoMeiVtasjLsmfSHRLZXT3QxwOyLXJaN1QWuDn4Nvpqie+OCLRXK2EYadjCR3blL5MPJVnBeqUDu4d8vDvaqN
red3Ax0MY5j/UT6ETOGPAF/NzJy1rci4wZ0aCS4omiY+l9MDQ62hYfbyqz+4nvfbVsKgHOm8RWmc+0EwkeL8Wk4nrEVs6ym2NrBp2cYU
flwKXY5/hUQmDiyTzAZdF1lquYYqMwu2tmC9WuuYHiC84pAKWG1K/wAJP6qVD79hV6zVMupw7Ws8dcXB/fFSKbvL9JHFOguDpVO1140K
MhLxv2rQH8UJKvQGw10er6xapRjIK0ptKfuD6/5+Pr8ceyMV4KXgDsmtAPU6mOlGoTXxjXnM8jKgFqnHBKLPCms+JnJIeaW2gEpHlGFZ
TPv54izrF3W7ml/JMF75GGX+GvSPMyhLhkzf4qpT/ihz3aAUpweVy3H1VJTgukyIBqgN8seR/Y/JbgVu3oiL9OCRdfkleQVHCyfwKHFi
dWa04vhhhACC5dwl3s+a8r+bgrIDQwKVKZjE0IKZ3KFzPkeWy6us92DcXY/9/x6fl95tFKhfiMfj4w/awQ5MWfWaeHQhSsQ9iRQJpfXr
v4HWam5aIfvIcCSbu1sD5ZDggzpVwramj4tivhDC3RMY6VupWSB0H0hmt+26zu37DsgU/xBGPP/TwT0Eycn0+IKTYSkwfDibQG3cJYee
WRiVZrLravrGv29DCxXD9UghWPYJlwrVkr8PrU89ADDlFu1fW3XZA+eooZhvGfUp5v8h1vznWUq7+b4owq3fsUxk5k/Tnze7OcmUgE5u
wwjBc+SiO8k2p3Bbnl0Z6sPDvILxNUwkBYdgE5gmSDxN12EiWDsgT76DdD7JqkYiBnuDgfymHFSZltSLSBBGs1K1uUvZ1UkSBOPDNrxb
5FeAmDAFe0xAs7FSWo6oV9z7FsVLne9DcYWPKCcbw5ansZKujZTimukblF2+YfeXCYjrtD7PXJLpvlgEGR8Zwb8qQdWxUTsyQCIVSIFo
6dpYjmeNTzL57BEkVpyhLPa8RJglRXmT5VtFYL8KE7S2azwDzAazd+RjpIz3MtCCfacS7gQVqPFr+4DEDMMnOiunIdaQRtCnm1g+Zkav
UUmQ/UiXH8s5mFUcUsR46isEx06mR+UNEA44ckxWHTmhGj7FwgqrzSvIsYzJt93B1vjSi2OeXOrW6qYgWfUHqH2M99BfZwk18ugHOWgO

```
2WhLG6KecECyQg1msHpk='}}] additional_kwargs={} response_metadata={'finish_reason': 'STOP', 'model_name':
'gemini-3.6-flash', 'safety_ratings': [], 'model_provider': 'google_genai'} id='lc_run--01a094b9-17d8-
7043-af36-cf9704eae962-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 13,
'output_tokens': 703, 'total_tokens': 716, 'input_token_details': {'cache_read': 0},
'output_token_details': {'reasoning': 465}}
```

What just happened: `ChatGoogleGenerativeAI` wraps the Gemini API behind LangChain's standard chat-model interface (`.invoke()`, `.stream()`, `.batch()`) — the same interface every other chat model in this notebook uses. That's the "consistent wrapper across providers" idea from section 1.1: nothing else in this notebook has to change if you swap `llm` for a different provider's chat model.

One detail worth remembering: `llm.invoke(...)` returns an `AIMessage` object, not a plain string. Above, we pulled the text out with `.text`. Later in this notebook you'll see two other common ways to get plain text out of a response: reading `.content` directly, or passing the message through `StrOutputParser()` inside an LCEL chain. All three get you to the same place — plain text you can print, store, or pass along.

1.3 Prompt Templates and LCEL

Checkpoint: what are we adding, and why?

Now we're adding structure to what was, a moment ago, just a plain string prompt. Instead of writing a brand-new instruction by hand every time, we'll build a reusable **prompt template** that accepts different inputs, and connect it to the model using **LCEL** (LangChain Expression Language) — the `|` operator you'll see below.

LCEL lets us pipe components together into a sequence: a prompt, a model, and an output parser become one reusable object. This is the pattern you'll rely on for the rest of the notebook: an LLM application isn't one giant block of code, it's small, connectable pieces.

After this section, you should be able to: build a `PromptTemplate` with named input variables, connect it to `llm` and `StrOutputParser()` with `|`, and call `.invoke()` on the resulting chain with a dictionary of inputs.

In [4]:

```
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

summary_template = PromptTemplate(
    input_variables=["text"],
    template=(
        "Summarize the following text in 2-3 sentences.\n\n"
        "Text: {text}\n\n"
        "Summary:"
    ),
)

# LCEL chain: prompt -> llm -> string parser
summary_chain = summary_template | llm | StrOutputParser()

sample = (
    "The belief that soft drinks directly cause urinary tract infections (UTIs) is a common myth."
    "UTIs are usually caused by bacteria entering and multiplying in the urinary tract, not by drinking
    "However, drinking large amounts of soft drinks may be indirectly related to urinary health."
    "Many soft drinks contain high amounts of sugar, which can contribute to unhealthy dietary habits w
    "Caffeinated soft drinks may increase urination in some people, although caffeine does not directly
    "People who drink a lot of soda may sometimes drink less water, which can reduce their overall flui
    "Drinking enough water helps increase urination, which may help flush bacteria from the urinary tra
    "Carbonated or caffeinated beverages can also irritate the bladder in some people and cause urinary
    "Bladder irritation can cause symptoms such as urgency or frequent urination, but these symptoms do
    "Therefore, soft drinks do not directly cause UTIs, but excessive consumption may be associated wit
)

print(summary_chain.invoke({"text": sample}))
```

Out [4]:

```
Soft drinks do not directly cause urinary tract infections, as UTIs are caused by bacteria entering and
multiplying in the urinary tract. However, excessive consumption of soft drinks can indirectly impact
urinary health by reducing overall water intake needed to flush out bacteria. Additionally, the caffeine
and carbonation in these beverages can cause bladder irritation and discomfort, which may mimic UTI
symptoms.
```

In [11]:

```
summary_template = PromptTemplate(
    input_variables=["text"],
    template=(
        "answer me like a toddler\n\n"
        "Text: {text}\n\n"
        "Summary:"
    ),
)

# LCEL chain: prompt -> llm -> string parser
summary_chain = summary_template | llm | StrOutputParser()

sample = (
    "The belief that soft drinks directly cause urinary tract infections (UTIs) is a common myth."
    "UTIs are usually caused by bacteria entering and multiplying in the urinary tract, not by drinking
    "However, drinking large amounts of soft drinks may be indirectly related to urinary health."
    "Many soft drinks contain high amounts of sugar, which can contribute to unhealthy dietary habits w
    "Caffeinated soft drinks may increase urination in some people, although caffeine does not directly
    "People who drink a lot of soda may sometimes drink less water, which can reduce their overall flui
    "Drinking enough water helps increase urination, which may help flush bacteria from the urinary tra
    "Carbonated or caffeinated beverages can also irritate the bladder in some people and cause urinary
    "Bladder irritation can cause symptoms such as urgency or frequent urination, but these symptoms do
    "Therefore, soft drinks do not directly cause UTIs, but excessive consumption may be associated wit
)

print(summary_chain.invoke({"text": sample}))
```

Out [11]:

```
Soda doesn't give you bad pee-pee bugs! Bad bugs do that, not soda.

But if you drink too much soda, you might forget to drink yummy water. Water is like a big slide that
flushes the yucky bugs out when you go potty! Soda can also make your bladder a little grumpy and make
you wanna pee fast-fast, but that's just a grumpy bladder, not a bug.

So soda doesn't cause the bad bug hurt, but drinking water makes your pee-pee super happy! 🥤💦
```

⚠ What just happened?

```
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

- `PromptTemplate` — a safe, parameterized template for prompts. It prevents the kind of bugs you get from concatenating strings by hand, and it validates that you've supplied every variable the template expects.
- `StrOutputParser` — takes the model's raw output (an `AIMessage`) and returns a clean string. Making this step explicit — rather than hiding it inside some larger object — is one of the things that makes LCEL chains easy to reason about: you can see exactly what each stage does to the data as it flows through.

```
summary_template = PromptTemplate(
    input_variables=["text"],
    template=(...),
)
```

- Creates a prompt template with one slot, `text`. The template string contains `{text}` as a placeholder; when you run the chain, LangChain checks that you actually provided `text` and substitutes it safely.

```
summary_chain = summary_template | llm | StrOutputParser()
```

- The `|` operator is **LCEL composition**: it pipes the rendered prompt into the model, then pipes the model's output into the string parser. Each piece — the template, the model, the parser — is a `Runnable`, and piping `Runnable`s together produces a new `Runnable`. That's the whole idea behind LCEL: build small, composable pieces, then connect them into a pipeline.
- The result, `summary_chain`, is a reusable "summarize" component: pass text in, get a summary out.

```
print(summary_chain.invoke({"text": sample}))
```

- Fills `{text}` with `sample`, renders the final prompt, sends it to the model, parses the output, and prints the summary. `.invoke(...)` is the standard entry point for any `Runnable` — the same object also gives you `.batch()` (run over many inputs) and `.stream()` (get tokens as they arrive) for free.

Why this pattern is useful:

- **Safety & clarity:** `PromptTemplate` enforces that required variables are provided and avoids brittle f-strings or string concatenation.
- **Reusability:** you can reuse `summary_chain` anywhere — an API, a batch job, a UI — by just swapping the input.
- **Separation of concerns:** prompt wording stays separate from calling code, which makes iteration, A/B testing, and localization easier.
- **Composability:** because any `Runnable` can be piped into any other, chains can be built up step by step (e.g., summarize → critique → revise) without extra glue code.

Prompts vs. Chains vs. "Pipelines"

Prompts (LangChain)

- Just the instruction text with placeholders (e.g., `{text}`, `{tone}`).
- Encourage consistent formatting, guard against missing variables, and standardize few-shot examples.

Chains (LangChain)

- Orchestrations that connect components: Prompt → LLM → (optional) parser → (optional) tool call, and so on.
- The simplest chain is an LCEL pipe: `prompt | llm`. More complex chains branch, loop, call tools, or combine multiple steps — expressed as LCEL `Runnable` compositions, or as a LangGraph graph once you need loops, retries, or multi-agent branching.
- "Runnables" (`prompt | llm | parser`) *are* how chains are built in LangChain today. Conceptually, it's still the same idea as any chain: a dataflow of steps.

Pipelines

- The word "pipeline" is more general, and commonly associated with Hugging Face's `transformers.pipeline(...)` or generic ML workflows (ingest → preprocess → model → postprocess).
- HF pipelines hide model/tokenizer/plumbing behind a single callable (e.g., `pipeline("summarization")`).
- Contrast:
  - HF pipelines are task-centric and opinionated (great for quick starts).
  - LangChain chains are framework-centric and composable, built to integrate prompts, tools, memory, retrieval (RAG), and multi-step logic across many LLM providers.

Common gotchas & tips

- **Make sure `llm` is defined:** in any LCEL chain, `llm` must be created earlier — a hosted chat model like `ChatGoogleGenerativeAI`, or a local Hugging Face pipeline if you're working with an on-device model.
- **Parsing outputs:** for structured results (JSON, bullet lists), add an output parser (e.g., `StrOutputParser()` or `JsonOutputParser()`).
- **Batching/streaming:** Chains/`Runnable`s support `.batch()` and `.stream()` out of the box — helpful for UIs, and no separate callback-based streaming setup is needed.
- **Check package boundaries when an import fails:** `langchain-core` holds the base abstractions (`PromptTemplate`, `Runnable`, output parsers). Provider integrations (`langchain-google-genai`, `langchain-huggingface`, `langchain-openai`, `langchain-anthropic`) and `langchain-community` hold everything else. A failed import is often just living in a different package than you expect.

1.4 Embeddings for Retrieval

Checkpoint: what are we adding, and why?

So far, our model could generate text, but it had no way to search our own information. This time, we're adding **embeddings** — a way of representing text numerically based on its meaning. Don't think of an embedding as a unique ID for a piece of text, and don't think of it as being based on word count; think of it as a mathematical representation that lets us compare pieces of text by *semantic similarity*. That distinction matters in the next section, where we use these vectors to help an application find relevant information before asking the LLM to answer.

It's also worth keeping the embedding model conceptually separate from the chat model you just used: `llm` generates text, while an embedding model like `GoogleGenerativeAIEmbeddings` converts text into a vector of numbers. They do different jobs and are used in different places in the pipeline.

After this section, you should be able to: explain, in your own words, why two pieces of text with similar meaning end up with similar embedding vectors, even if they don't share many of the same words.

In [5]:

```
import os
from dotenv import load_dotenv
from langchain_google_genai import GoogleGenerativeAIEmbeddings

# Load the same Google AI Studio API key used by the chat model
load_dotenv()

if not os.getenv("GOOGLE_API_KEY"):
    raise ValueError(
        "Set GOOGLE_API_KEY in the environment or a local .env file before running this cell."
    )

# Gemini embedding model
emb = GoogleGenerativeAIEmbeddings(
    model="gemini-embedding-001"
)

# Convert text into an embedding vector
vec = emb.embed_query("How to deploy a Streamlit app?")

print("Vector length:", len(vec))
```

Out [5]:

```
Vector length: 3072
```

Embeddings are the building block behind semantic search and retrieval-augmented generation (RAG) — the pattern you'll build in the next section.

Embedding model	Typical vector dimensions
Gemini `gemini-embedding-001`	**3,072**
OpenAI `text-embedding-3-small`	**1,536**
OpenAI `text-embedding-3-large`	**3,072**
`all-MiniLM-L6-v2`	**384**
Some other embedding models	768, 1,024, 1,536, 2,048, etc.

Section 2 — Use Cases by Autonomy Level

We'll organize LLM applications by **autonomy**: how much the AI acts beyond simply generating a response. As you move through the levels below, notice that each one keeps everything from the level before it and adds exactly one new capability on top.

Level 1 — Basic LLM Apps (no autonomy)

Checkpoint: what are we adding, and why?

Now we stop simply testing the LLM and start building an actual application around it. Notice this isn't a general-purpose chatbot — we're giving the model one specific job: rewrite text in a requested tone. An LLM application can be designed around a particular task rather than an open-ended conversation.

At this level, **there is no autonomy**: we, the developers, decide exactly what happens and in what order.

`input → prompt template → LLM → output`

The model isn't deciding what tools to use or what action to take — it's filling in a fixed template. This gives us a baseline for understanding what changes once we add retrieval, memory, and eventually decision-making in the sections that follow.

Behavior: single-turn prompt → response. No tools, no memory.

Use cases:

- Ad-hoc Q&A and drafting (emails, posts, summaries)
- One-off data transformation (rewrite text, translate tone)

Analogy: like a smart typewriter — you ask, it rewrites/summarizes/drafts, and it's done. It doesn't remember past inputs or plan future steps.

After this section, you should be able to: describe why a rephrasing chain like the one below counts as "no autonomy," even though it's a working LLM application.

In [6]:
```
# Rephrase text with a specific tone (Level 1 example)
rephrase_template = PromptTemplate(
    input_variables=["text", "tone"],
    template=(
        "Rewrite the text below in a {tone} tone while preserving meaning.\n"
        "Text: {text}\n\nRewritten:"
    ),
)

rephrase = rephrase_template | llm | StrOutputParser()

print(rephrase.invoke({"text": "Please join our meeting at 3 PM.", "tone": "friendly"}))
```

Out [6]:
```
Hope you can join us for our meeting at 3 PM!
```

Level 2 — Co-Pilots (human-in-the-loop)

Behavior: multi-turn assistance that can retrieve context (RAG), keep short-term memory, and suggest steps — with a human still reviewing the result.

Use cases:

- Coding/document co-pilots
- PDF/KB Q&A assistants (retrieve first, then answer)
- Draft → review → apply workflows

LangChain pattern: Prompt → LLM → Parser, combined with retrieval (a vector store) and, optionally, memory — expressed as an LCEL retrieval chain, optionally wrapped with LangGraph for memory.

Example 1: Retrieval-Augmented Q&A (RAG)

Checkpoint: what are we adding, and why?

This time, we're adding something our Level 1 application did not have: access to external knowledge. Instead of relying only on what the LLM already knows, we're giving the application a knowledge base it can search. You'll see the complete retrieval process end to end:

```
Question
   ↓
Retriever
   ↓
Relevant Documents
   ↓
format_docs()
   ↓
Context
   ↓
PromptTemplate
   ↓
LLM
   ↓
Answer
```

Here's what each piece actually does — worth keeping straight, since it's easy to blur them together:

- **`docs`** — the knowledge/document content itself.
- **`emb`** — the embedding model, which turns text into vectors.
- **`FAISS`** — the vector store that holds those vectors so they can be searched.
- **`retriever`** — the component that searches the vector store and returns the most relevant documents for a given question.
- **`format_docs()`** — a helper function that converts the retrieved `Document` objects into plain, readable text. It's a formatting step, not a database — the vector store is `FAISS`.
- **`PromptTemplate`** (`rag_prompt` below) — structures the instructions and the retrieved context into a single prompt for the LLM.
- **`llm`** — generates the final answer from that prompt.

This example uses a small, hardcoded knowledge base — just two `Document` objects — for demonstration purposes. The pattern itself is the point: in a real application, those documents could come from PDFs, websites, a database, or a company's internal files, but the retrieval flow above wouldn't change. With this pattern, you can start building document Q&A systems, company-policy assistants, FAQ systems, and any application that needs to answer questions using private or specialized information.

After this section, you should be able to: trace a single question through the diagram above and name which line of code corresponds to each arrow.

In [7]:

```
import re
from operator import itemgetter

from langchain_core.documents import Document
from langchain_community.vectorstores import FAISS
from langchain_core.runnables import RunnableLambda

# 1) Tiny KB + retriever
docs = [
    Document(
        page_content="Refunds: Within 30 days = full refund; after 30 days = store credit.",
        metadata={"source": "policy_v1"},
    ),
    Document(
        page_content="No refunds for custom-made items.",
        metadata={"source": "policy_v1"},
    ),
]

vs = FAISS.from_documents(docs, emb)
retriever = vs.as_retriever(search_kwargs={"k": 2})

def format_docs(dlist):
    return "\n".join(f"- {d.page_content} [source: {d.metadata.get('source', '')}]" for d in dlist)

# 2) Tight prompt (don't give the model text you don't want echoed in the answer)
rag_prompt = PromptTemplate.from_template(
    "You are a precise assistant.\n"
    "Use ONLY the context to answer the question. If the answer is not present, reply exactly: I don't
    "Return ONLY the final answer as one short sentence. Do NOT include source tags, the word 'Response
    "any instruction text.\n\n"
    "Context:\n{context}\n\nQuestion: {question}\nFinal answer:"
)

# 3) Post-cleaner to strip leftovers and keep first sentence
def clean_answer(txt: str) -> str:
    low = txt.lower()
    i = low.find("final answer:")
    if i != -1:
        txt = txt[i + len("final answer:"):].strip()
    txt = re.sub(r"\s*\[source:[^\]]*\]\s*", " ", txt)
    txt = re.sub(r"(?i)\bresponse:\s*", "", txt)
    txt = re.sub(r"(?is)if the answer is not present.*$", "", txt).strip()
    txt = re.sub(r"\s+", " ", txt).strip()
    m = re.match(r"(.+?[.!?])(\s|$)", txt)
    if m:
        txt = m.group(1).strip()
    return txt

# 4) LCEL chain: retrieve -> format -> prompt -> llm -> parse -> clean
#    (StrOutputParser handles the AIMessage -> str conversion for us here, so no manual
#    .content extraction is needed in this particular chain)
rag_chain = (
    {
        "context": itemgetter("question") | retriever | format_docs,
        "question": itemgetter("question"),
    }
    | rag_prompt
    | llm
    | StrOutputParser()
    | RunnableLambda(clean_answer)
)

# 5) Try it
print("Q1:", "What is the refund policy for custom-made items?")
```

```
print("A1:", rag_chain.invoke({"question": "What is the refund policy for custom-made items?"}))

print("\nQ2:", "How are refunds handled after 30 days?")
print("A2:", rag_chain.invoke({"question": "How are refunds handled after 30 days?"}))
```

Out [7]:

```
C:\Users\manda\AppData\Local\Temp\ipykernel_17792\2862831100.py:5: DeprecationWarning: `langchain-community` is being sunset and is no longer actively maintained. See https://github.com/langchain-ai/langchain-community/issues/674 for details and migration guidance toward standalone integration packages.
  from langchain_community.vectorstores import FAISS
Q1: What is the refund policy for custom-made items?
A1: There are no refunds for custom-made items.

Q2: How are refunds handled after 30 days?
A2: After 30 days, refunds are issued as store credit.
```

Using a PDF/KB Chatbot in Practice

A common real-world version of this same pattern is a PDF chatbot — a flow like the one below, adapted from Abdullah Abdul Wahid's "*Langchain Chatbot for Multiple PDFs*" (Medium, Jun. 18, 2023):

 Flow diagram: PDF leads to Documents; User leads to Question; both feed into a Prompt bubble, which goes to an OpenAI model icon, which produces a Response

Flow: PDF → split into Documents → combined with the user's Question → assembled into a Prompt → sent to the model → Response returned.

(The diagram shows an OpenAI model specifically, since that's what the article it's adapted from used — the same flow applies with any chat model, including the Gemini model used throughout this notebook.)

Usage, step by step:

- **Upload PDF documents:** a sidebar/UI lets a user upload one or more PDF files, so the app can draw from a range of PDF sources for retrieval.
- **Ask questions:** once documents are uploaded, the user enters questions related to the uploaded content; the app uses NLP techniques to interpret the query.
- **Receive answers:** the app processes the extracted PDF content, using retrieval plus the language model to generate a context-aware, grounded response.

The `rag_chain` above *is* this pattern, minus the file-upload UI and PDF parsing (see LangChain's `PyPDFLoader`, or the `pdf` skill in this environment, if you want to load real PDFs instead of the two-sentence toy KB used here).

Example 2: Conversation Memory (multi-turn)

Checkpoint: what are we adding, and why?

Our RAG application can now find information, but it still doesn't naturally remember the conversation. Each call to `rag_chain` above is independent — ask a follow-up question with "it" or "that," and the chain has no idea what you're referring to. This time, we're adding **conversation memory** so the application can keep track of previous turns.

The important idea here: "memory" isn't something that magically exists inside the LLM itself. The model has no built-in recollection of your previous message — our application has to manage and supply the relevant conversation history on every single call. We do that here with LangGraph: a `StateGraph` compiled with a checkpointer, which stores and replays messages per conversation automatically.

With this capability, you can start building applications that feel much more like real assistants — for example, a support assistant that understands follow-up questions without making the user repeat themselves.

After this section, you should be able to: explain what would happen to the second question below (`"And what about custom-made items?"`) if you removed `history` from the prompt entirely.

In [8]:

```
from langchain_core.messages import HumanMessage, AIMessage
from langchain_core.output_parsers import StrOutputParser
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.memory import InMemorySaver

# 1) Chat prompt with a history slot
chat_rag_prompt = PromptTemplate.from_template(
    "You are a precise assistant. Use ONLY the provided context when answering.\n"
    "If the answer is not present, reply exactly: I don't know.\n"
    "Return only the final answer as one short sentence.\n\n"
    "Conversation so far:\n{history}\n\n"
    "Context:\n{context}\n\nQuestion: {question}\nFinal answer:"
)

def format_history(messages):
    lines = []
    for m in messages:
        role = "User" if isinstance(m, HumanMessage) else "Assistant"
        lines.append(f"{role}: {m.content}")
    return "\n".join(lines)

# 2) Node: retrieve -> prompt -> Gemini chat model -> parse -> clean
def call_model(state: MessagesState):
    question = state["messages"][-1].content
    history = format_history(state["messages"][:-1])
    context = format_docs(retriever.invoke(question))

    prompt_text = chat_rag_prompt.format(
        history=history,
        context=context,
        question=question
    )

    raw = llm.invoke(prompt_text)

    # Convert Gemini's response into a plain string
    answer = StrOutputParser().invoke(raw)

    # Clean the final answer
    answer = clean_answer(answer)

    return {"messages": [AIMessage(content=answer)]}

# 3) Build the graph and attach a checkpointer
workflow = StateGraph(state_schema=MessagesState)
workflow.add_node("model", call_model)
workflow.add_edge(START, "model")

memory = InMemorySaver()
app = workflow.compile(checkpointer=memory)

# 4) Per-session config
config = {"configurable": {"thread_id": "user-123"}}

# 5) Multi-turn examples
r1 = app.invoke(
    {"messages": [HumanMessage(content="What is the refund policy?")]},
    config
)
print(r1["messages"][-1].content)

r2 = app.invoke(
    {"messages": [HumanMessage(content="And what about custom-made items?")]},
    config
```

```
)
print(r2["messages"][-1].content)
```

```
Out [8]: Full refunds are provided within 30 days, store credit is given after 30 days, and custom-made items are non-refundable.
Custom-made items are non-refundable.
```

What just happened? Comparing the two examples

- **Example 1: Retrieval-Augmented Q&A (RAG)** — single-turn, no memory.
- **Example 2: Conversation Memory** — multi-turn, with history injected into the prompt on every call.

What stays the same

- **Retriever and knowledge base:** both examples use the same FAISS retriever and `format_docs(...)`. You're still grounding answers on retrieved context, so the model is less likely to invent an answer.
- **LLM:** same model and generation settings.
- **Post-cleaning:** both examples run the model's output through `clean_answer(...)` to trim stray echoes or repeated instructions.

What's different, and why it matters

1) Prompt/state shape. `PromptTemplate` with a plain `{context}` / `{question}` slot (Example 1) vs. a prompt that also renders `{history}`, sourced from LangGraph's `MessagesState` (Example 2).

- In single-turn RAG, each call is independent — there's no memory of prior Q&A.
- With conversation memory, `MessagesState` accumulates the conversation automatically, and `format_history(...)` renders it into the prompt — this is what lets the model resolve a follow-up like "And what about custom-made items?"

2) Memory orchestration. None (Example 1) vs. a **LangGraph checkpointer** (Example 2). `InMemorySaver()` (swap for a Postgres/Redis-backed saver in production) is attached to a compiled `StateGraph`. The graph stores and replays messages per `thread_id` automatically.

3) Multi-turn behavior. Isolated questions (Example 1) vs. context-aware follow-ups (Example 2) — the second turn in Example 2 relies on the first answer's topic because history is now part of the prompt.

4) Input schema & invocation. `rag_chain.invoke({"question": ...})` (Example 1) vs. `app.invoke({"messages": [HumanMessage(...)]}, config)` (Example 2), where `config` carries the `thread_id` that ties a sequence of turns together.

Pitfall to avoid: if you call the *un-compiled* `call_model` function directly instead of going through `app.invoke(...)`, you lose persistence entirely — always call the compiled, checkpointed graph.

Summary

Dimension	Example 1: RAG (single-turn)	Example 2: Conversation Memory (multi-turn)
Prompt class	`PromptTemplate`	`PromptTemplate` + rendered `{history}`
Memory	None	LangGraph `StateGraph` + `InMemorySaver` checkpointer
Inputs	`question`	`messages` (LangGraph injects prior history via `thread_id`)
Behavior	One-shot Q&A grounded on retrieved context	Multi-turn, context-aware Q&A grounded on retrieved context + chat history
Best for	FAQs, search-like queries, batch jobs	Assistants/co-pilots, follow-ups, pronouns & ellipses ("that", "those", "it")

When to choose which

- **Use single-turn RAG** when queries are independent (FAQ bots, batch processing, API endpoints where each call is stateless).
- **Use conversational RAG with memory** when you expect follow-ups, clarifications, and incremental tasks (coding/doc co-pilots, helpdesk chats).

Reference: How LangChain's Core Concepts Fit Together

Objective: to provide a map of the concepts underlying LangChain, and where each one shows up in the application you've been building above.

LangChain is a framework designed to enhance the capabilities of large language models through modular components. These components let you build, manage, and deploy sophisticated language model applications by combining small, well-defined pieces rather than writing everything from scratch.

It's useful to think about LangChain in terms of seven conceptual areas. A few of them — Memory and Indexes, in particular — aren't separate things you import anymore; they've folded into retrieval and LangGraph persistence, which you've already used above. The concepts are still worth knowing by name, because you'll see them referenced this way in documentation, tutorials, and other people's code:

1. Models
2. Prompts
3. Memory
4. Indexes
5. Chains
6. Agents
7. Callbacks

 LangChain's six interconnected modules arranged in a circle around a central LangChain node: Models, Prompts, Indexes, Chains, Agents, Memory

Six of the seven areas above, shown as they relate to each other (Callbacks isn't pictured). As you'll see below, Indexes and Memory are best understood today as part of retrieval and LangGraph persistence respectively, and Agents has grown into one of the most heavily used parts of the framework.

1. Models

Purpose: the Models area covers integrating different types of language models — LLMs, chat models, and text embedding models.

Applications:

- **LLMs (Large Language Models):** generate human-like text based on a given prompt.
- **Chat Models:** handle conversational, multi-turn AI tasks.
- **Text Embedding Models:** convert text into numerical vectors for similarity search and clustering.
- **Example use case:** using a hosted model like Gemini to generate coherent, contextually relevant content — exactly what you did in section 1.2 above.

In practice, working with "Models" today means picking a provider integration package — `langchain-google-genai`, `langchain-huggingface`, `langchain-openai`, `langchain-anthropic`, and so on — and using that provider's chat, LLM, or embedding class behind LangChain's shared interface.

2. Prompts

Purpose: designing and managing the prompts fed into language models, so they're structured in a way that maximizes the accuracy and consistency of the model's output.

Applications:

- Creating structured prompts for specific tasks.
- Managing dynamic prompts that change based on context.

Example use case: designing a prompt to summarize a given text, or to answer specific questions based on retrieved context — exactly what `PromptTemplate` does in sections 1.3 and Level 2 above.

This is exactly what `PromptTemplate` / `ChatPromptTemplate` do, living in `langchain_core.prompts` and typically used as the first stage of an LCEL pipe.

3. Memory

Purpose: letting models retain information across interactions — essential for applications that need context to persist across multiple queries.

Applications:

- Building chatbots that remember user interactions.
- Systems that need to recall previous inputs for a continuous task.

Example use case: a customer service chatbot that remembers previous interactions with a user to give more personalized responses — the pattern you built in the Conversation Memory example above.

If you come across older tutorials that build memory with classes like `ConversationBufferMemory`, know that the current recommended approach is **LangGraph persistence** — a `StateGraph` with a checkpointer, exactly as demonstrated earlier in this notebook.

4. Indexes

Purpose: creating and managing indexes over large text corpora, for efficient retrieval and similarity search.

Applications:

- Indexing documents for quick search and retrieval.
- Building recommendation systems based on text similarity.

Example use case: indexing a set of research papers to quickly find relevant documents for a query — exactly what `FAISS.from_documents(docs, emb)` did for our tiny refund-policy knowledge base above.

Today this area is simply called **retrieval**: vector stores (like the `FAISS` store used above) paired with retrievers. The concept and the use case are the same as "Indexes" — only the name changed.

5. Chains

Purpose: creating complex workflows by chaining components together, so the output of one step becomes the input to the next.

Applications:

- Multi-step data processing pipelines.
- Automating complex decision-making processes.

Example use case: a workflow that takes user input, processes it through a series of models, and produces a final, actionable output.

Every `prompt | llm | parser` construction in this notebook is a chain in exactly this sense — LCEL is the current, direct way to build one.

6. Agents

Purpose: creating autonomous components that can make decisions and take actions based on the data they process, rather than following one fixed sequence of steps.

Applications:

- Autonomous systems for monitoring and alerting.
- Intelligent agents for automated customer support.

Example use case: an agent that monitors for a certain kind of query and decides, on its own, whether it needs to look something up before responding — see the Level 3 — Agents section below for a working example built with `create_agent`.

Agents, built on LangGraph, are one of the most actively used parts of LangChain in production today.

7. Callbacks

Purpose: triggering functions at specific stages of a processing pipeline — useful for logging, monitoring, and extending functionality.

Applications:

- Logging intermediate steps for debugging.
- Triggering custom actions during model processing.

Example use case: logging the input and output of each step in a text-generation pipeline for audit purposes.

Callbacks still work exactly as described above. In production, this role is increasingly handled by **LangSmith**, LangChain's observability/tracing platform, which wraps callback-level detail into a hosted dashboard rather than requiring you to wire up callback handlers by hand.

LangChain's modular design offers flexibility and scalability for building advanced language processing applications. Understanding and combining these seven areas is what lets you go from "call a model" to the kind of applications you're building in this notebook.

LangChain — Model Modules Concept

The Models area of LangChain covers integrating different types of language models. As discussed above, when we talk about models in LangChain, we're primarily talking about three types:

 Diagram: a Prompt block (Task Description, Current Input, Output Indicator) feeds as Input into a Large Language Model box, which returns Output as Generated Text

1. LLMs (Large Language Models)

- **Functionality:** LLMs take prompts as input and generate text completions. Trained on vast amounts of data, they produce outputs suitable for a wide variety of tasks.
- **Applications:** generating content on a topic, explaining a process, or any other text generation task.
- **Choosing an LLM:** the choice depends on performance, cost, ease of use, and flexibility. Hosted options like Gemini offer strong performance with minimal setup; open-source options available through Hugging Face offer more flexibility and community support, at the cost of needing your own compute.

2. Chat Models

- Designed specifically for conversational AI tasks, chat models handle dialogue and multi-turn interaction. LangChain's chat model interface (`BaseChatModel`) is the standard way to work with multi-turn, role-based conversation — it's what `ChatGoogleGenerativeAI` implements above, and what `create_agent` builds on in the Agents section below.

Text Embedding Models

Convert text into numerical vectors, which are essential for tasks like similarity search and clustering.

 Diagram: Words, Paragraph, and Articles all feed into a robot icon (the embedding model), which outputs a column of numerical vector values

As Harsh Bakshi puts it in "*Text Embedding Models In LangChain*" (Medium, Aug. 21, 2023): text embedding models turn natural language into numerical representations that capture meaning, structure, and context — representations that power semantic search, text analysis, and text generation. Working with embedding models directly can require specialized knowledge; LangChain's contribution is a simple, consistent interface for building on top of embedding models from different providers, without needing to handle the low-level technical details yourself.

Variety of text embedding models in LangChain

LangChain offers a wide range of text embedding integrations, each with its own tradeoffs — including Gemini's own embedding models (used above), OpenAI, Cohere, Hugging Face Hub, and others. Most ship as separate provider packages (`langchain-google-genai`, `langchain-openai`, `langchain-huggingface`, and so on) rather than inside the core `langchain` package.

Practical insight: to explore embedding models beyond the one used in this notebook, Hugging Face is a good place to start — it offers a large community and tooling for building, training, and deploying models. Many public models, like `sentence-transformers`, require no API key at all; a key only matters for gated or private models.

Level 3 — Agents

Checkpoint: what are we adding, and why?

This is where we introduce something fundamentally different from every previous stage: decision-making about what action to take. In the RAG example, *we* designed the workflow: retrieve → format → prompt → LLM, in that fixed order, every time. Now, we're giving the model access to a tool and letting the agent itself decide when that tool is useful.

Notice what we're *not* doing: we're not throwing away the RAG system we already built. We're turning its retrieval capability into a **tool** that the agent can choose to call. This is the beginning of **agentic AI** — applications that choose between available actions rather than following one predetermined path. From here, you can build applications that use multiple tools, retrieve information only when it's actually needed, take actions, and coordinate several steps toward a goal.

The key contrast:

RAG chain — the developer explicitly defines the sequence:

```
Question
→ Retriever
→ Context
→ Prompt
→ LLM
→ Answer
```

Agent — the agent decides what to do at each step:

```
Question
→ Agent
→ decides whether it needs the tool
→ Tool (if needed)
→ Agent
→ Answer
```

What makes the tool below work: the agent doesn't magically have access to the documents. The `@tool`-decorated function contains the actual code that calls the existing `retriever`:

```
@tool
refund_policy_lookup()
        ↓
retriever.invoke(query)
        ↓
FAISS
        ↓
relevant documents
        ↓
format_docs()
        ↓
tool result
```

In other words, the agent is **reusing the retrieval capability built in the RAG section** — it's just wrapped so the agent can decide, on its own, when to call it.

After this section, you should be able to: explain why the code below counts as "agentic" even though it calls the same `retriever` and `format_docs` you already built.

In [9]:

```
from langchain.agents import create_agent
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage
from langchain_core.output_parsers import StrOutputParser

@tool
def refund_policy_lookup(query: str) -> str:
    """Look up refund policy information relevant to the query."""
    return format_docs(retriever.invoke(query))

agent = create_agent(
    model=llm,
    tools=[refund_policy_lookup],
    system_prompt=(
        "You are a precise store-policy assistant. "
        "Use the refund_policy_lookup tool when needed. "
        "Answer in one short sentence."
    ),
)

result = agent.invoke({
    "messages": [
        HumanMessage(
            content="Can I get a refund on a custom item after 40 days?"
        )
    ]
})

# Convert the agent's AIMessage into plain text
answer = StrOutputParser().invoke(result["messages"][-1])

print(answer)
```

Out [9]:

```
No, custom-made items are not eligible for refunds.
```

A useful conceptual distinction to hold onto going forward:

> ***LCEL is useful when we explicitly define a sequence of operations.***

versus

> ***An agent can dynamically decide which tool or action to use.***

LCEL doesn't disappear once you start using agents — you'll often still use LCEL *inside* a tool (as `refund_policy_lookup` does here) or inside individual steps of a larger system. What changes is who decides the order of operations: you, or the agent.

Summary

Across this notebook, you built one application, piece by piece:

- **LLMs, chat models, and embeddings** as the three model interfaces you'll reach for.
- **Prompt templates** as a safety and reuse mechanism, so prompts aren't hand-written strings.
- **LCEL** as the way to compose those pieces into a pipeline — prompt → model → parser.
- **RAG** as the default pattern for grounding answers in your own data, using a vector store and a retriever.
- **Conversation memory**, managed explicitly by your application through LangGraph, not by the model itself.
- **Agents**, which reuse everything you built and add the ability to decide, at runtime, when a tool is actually needed.
- **Autonomy levels** — basic → co-pilot → agent — as a way to scope what a given project actually needs, rather than reaching for the most complex pattern by default.

Frameworks worth knowing about as you go further:

- **LlamaIndex** — strong for pure document retrieval/RAG pipelines; many teams use it alongside LangChain rather than instead of it.
- **LangGraph** — LangChain's own agent runtime, and effectively the production layer for anything stateful or multi-step.
- **CrewAI, Haystack, Semantic Kernel, DSPy** — alternative agent/orchestration frameworks worth a skim if you're comparing options for a specific project.